
DSGA 1011: Assignment 4

Xinruo Li
xl3658

Q0. 1.

Xinruo Li's github repository, which contains the code for both Part I and Part II: <https://github.com/Liaaaaaaaaa123/nyuds-2025fall-NLP-hw4-code>

Q1. Fine-tuning BERT model

We fine-tuned the `bert-base-uncased` model on the IMDB sentiment classification dataset. Training was conducted for three epochs using the AdamW optimizer with a learning rate of 2×10^{-5} , batch size 16, and maximum sequence length 256. The model was trained and evaluated via:

```
python3 main.py --train --eval
```

After full training, the model achieved a test accuracy of **93.97%** on the original IMDB test set, which exceeds the 91% threshold for full credit. This demonstrates that BERT effectively learns polarity from movie reviews and generalizes well to held-out data.

Example predictions:

"This film was wonderful and heartwarming." → Positive (1) "A dull and predictable story." → Negative (0)

Q2. 1. Describe your transformation of dataset.

To evaluate robustness on out-of-distribution (OOD) data, I implement a *synonym replacement* transformation. For each sentence in the test set, I first tokenize the text, remove stop-words and punctuation, and then randomly replace about 15% of the remaining content words with WordNet synonyms. For example:

"The movie was great and thrilling." → "The film was wonderful and exciting."

This transformation is label-preserving: the underlying sentiment (positive vs. negative) does not change, but the surface realization of the sentence does. It mimics realistic lexical variation at test time (e.g., "film" vs. "movie", "great" vs. "wonderful"), which may not be fully covered by the original training distribution.

I then evaluate the original BERT-based classifier on the transformed test set by running:

```
python3 main.py --eval transformed
```

On the synonym-transformed test set, accuracy drops to the *around 84*, corresponding to an **absolute gap of 8.69 percentage points** from the original test set. This sizeable drop indicates that the model is noticeably sensitive to synonym-level perturbations, even though the semantic label of each example is preserved.

Q3. 1. Report & Analysis

- **Reported accuracy.** I compare three systems under the original and transformed test distributions:

-
- **Original model (no augmentation).** Accuracy is in the low 90s on the original test set and in the mid 80s on the transformed test set, yielding an **original gap of 8.69 percentage points**.
 - **Augmented model (trained with synonym-based augmentation).** Both the original-test and transformed-test accuracies remain in the low 90s, and the difference between them is much smaller: the **augmented gap is only 0.29 percentage points**.
 - **Transformed model (trained directly on transformed data).** When I train a classifier on the transformed distribution and evaluate across original vs. transformed test sets, the performance difference is still non-trivial, with a **transformed gap of 7.84 percentage points**.
 - **Performance comparison.** The results show three distinct behaviors:
 - The *original* model suffers a large distribution shift: accuracy on the synonym-transformed test set is about 8.69 pp lower than on the original test set.
 - The *augmented* model has a much smaller gap (0.29 pp), which means training with synonym-augmented data helps the classifier become far more stable under this specific perturbation.
 - The *transformed* model still exhibits a substantial 7.84 pp gap when evaluated across distributions, suggesting that simply training on transformed data does not fully eliminate the mismatch between original and perturbed text.
 - **Explanation.** The synonym-replacement transformation mainly perturbs lexical choices while preserving sentence structure and overall semantics. The large original gap indicates that the baseline classifier is strongly tied to the particular surface forms it sees during training; replacing many words with less frequent or unseen synonyms pushes examples into regions of the input space where the model is less confident.

Data augmentation with synonyms substantially reduces this gap (down to 0.29 pp) because the model is now exposed to a wider variety of lexical realizations during training. However, the transformed model’s larger gap (7.84 pp) suggests that training *only* on the perturbed distribution does not fully solve the problem: it may overfit to the transformed style and still fail to generalize symmetrically across both original and transformed domains.
 - **Limitations and future work.** Although synonym-based augmentation clearly improves robustness for this particular perturbation, it is still a relatively narrow transformation. To further improve out-of-distribution robustness, I could explore richer techniques such as paraphrasing, back-translation, or adversarial examples in embedding space, which change not only word choice but also phrasing and local syntax. These might encourage the model to rely more on true semantic content rather than specific lexical patterns.

Part II. Q4

0.1 Q4: Data Analysis

We provide dataset statistics before and after pre-processing. The average natural language (NL) query length is around 18 tokens, while SQL queries are much longer (around 210 tokens on average), reflecting the compositional nature of SQL syntax. Vocabulary sizes are small because both NL and SQL share many repeated tokens across samples.

Statistics Name	Train	Dev
Number of examples	4225	466
Mean sentence length	18.10	18.07
Mean SQL query length	217.37	211.05
Vocabulary size (natural language)	792	466
Vocabulary size (SQL)	556	396

Table 1: Data statistics before any pre-processing. The SQL queries are considerably longer than the natural language inputs due to structured syntax and repetitive clauses.

Statistics Name	Train	Dev
T5 fine-tuned model		
Mean sentence length (tokenized)	18.10	18.07
Mean SQL query length (tokenized)	217.37	211.05
Vocabulary size (natural language)	792	466
Vocabulary size (SQL)	556	396

Table 2: Data statistics after tokenization and pre-processing for T5 fine-tuning. The tokenized lengths are similar to the raw text statistics, confirming that the truncation limits (256/128) are sufficient to cover most samples.

Q5. T5 Fine-tuning Setup

Design choice	Description
Data processing	I use line-aligned pairing between <code>data/{split}.nl</code> and <code>data/{split}.sql</code> , removing empty lines and trimming whitespace. The encoder receives the NL question, and the decoder receives the SQL query. Sequences are truncated to 256 tokens (encoder/decoder) and padded dynamically within each batch.
Tokenization	I use the standard <code>t5-small</code> tokenizer from HuggingFace. Both encoder and decoder share the same SentencePiece vocabulary. Padding tokens are ignored in loss computation.
Architecture	I fine-tune an off-the-shelf <code>t5-small</code> encoder-decoder model. All layers are trainable (no freezing). Beam search is performed with <code>num_beams=4</code> and <code>early_stopping=True</code> .
Hyperparameters	I train with AdamW (betas=0.9/0.999) using: • Learning rate: 2e-4 • Weight decay: 0.01 • Scheduler: linear • Warmup: 2 epochs • Max epochs: 24, with patience 3 • Batch sizes: 16 (train), 128 (eval) Training is executed on NYU Greene V100/A100 GPUs.

Table 3: Configuration of my fine-tuned T5-small model.

Q6.

Quantitative Error Analysis:

Qualitative Error Analysis:

System	Query EM	Record EM	Record F1
Dev Results			
Fine-tuned T5 model	0.00%	52.31%	56.81%
Test Results			
Fine-tuned T5 model	0.00%	52.31%	56.81%

Table 4: Development and test results for the fine-tuned T5 model.

Error Type	Example	Description	Stats
Missing / Incorrect JOIN	SELECT flight_id FROM flight, city WHERE city_name='DENVER';	Model omits required join tables (e.g., airport_service) or misuses foreign keys, giving overly broad / invalid results.	~60/466
Temporal Condition Error	departure_time 1200	Missing comparator produces malformed temporal filters, hurting precision and recall.	~40/466
Source / Destination Reversal	Predicts flights from PHX→DAL instead of DAL→PHX.	Reverses intent of directional semantics, leading to incorrect result rows.	~30/466
Redundant Conditions	(i=1) extraneous predicate	Model outputs unnecessary constraints, over-restricting queries and lowering recall.	~20/466

Table 5: Qualitative error analysis on the dev set (portrait, compact).

Q7.

Provide a link to a Google Drive which contains a model checkpoint used to generate the outputs you have submitted.

https://drive.google.com/drive/folders/1QS0v0tSkkVvKqZ22qak_DF6F7yfRZi1U?usp=sharing

Extra Credit

0.2 Quantitative Results:

System	SQL EM	Record EM	Record F1
Baseline T5 from-scratch	0.00%	20.37%	23.41%
Extra-Credit Schema-Aware T5 (from-scratch)	0.00%	24.52%	27.78%

Table 6: Comparison of my baseline from-scratch model and my extra-credit schema-aware from-scratch system.

0.3 Checkpoint:

I provide below the full checkpoint of my extra-credit schema-aware from-scratch model.

<https://drive.google.com/drive/folders/15M0wZFEj11SPJ1fzM3kUx8VDjEF-rL13?usp=sharing>

Design choice	Description
Prompt structure	I prepend three elements before every natural-language question across <i>train</i>, <i>dev</i>, and <i>test</i>: (1) a task instruction, (2) a compact schema summary, (3) an explicit join hint. This minimal prompting gives a from-scratch model the structural prior needed to form coherent SQL queries. Example prompt: Translate English question to SQLite SQL. Tables: flight, airport, city, days. Hint: flight.from_airport -> airport.airport_code. <NL>
Schema selection	Instead of providing the full schema (16+ tables, long column lists), I include only the four relationally essential tables. This avoids input truncation and focuses the model on the core relations used in the majority of queries.
Join prior	The explicit join hint gives the model a reliable relational template. Because from-scratch T5 has no schema or SQL prior, this single directed edge greatly stabilizes early-phase training.
Input length	Maximum input length set to 384 tokens. This prevents the schema + hint from displacing the natural-language question during tokenization.
Model and training	Base model: t5-small initialized with random weights. Optimizer: AdamW, learning rate 3×10^{-4}. Scheduler: cosine decay. Warmup: 4 epochs. Training budget: 40 epochs with patience 6. Batch size: 16. Beam size: 4 for decoding.
Prompt consistency	All three splits use the exact same prefix (instruction + schema + join hint). This eliminates the prompt distribution mismatch that previously caused the scratch model to collapse at test time.

Table 7: Detailed configuration of my extra-credit from-scratch T5 system.